TACHYON

AIML Intern Pre-Boarding Plan

Pre-Boarding Foundations + AI Concepts Brush-Up · v1.0 · 2026

Audience	Total Est. Time	Goal
Newly Hired AIML Interns	~60–80 hours total	Understand AI → build first RAG agent on cloud

This plan is sent to all AIML interns as soon as they accept their offer. It is split into two parts: (1) Pre-Boarding Foundations — the technical baseline every intern must have before Day 1, and (2) the AI Ramp Plan — an AI Concepts Brush-Up — 8 topics to familiarise yourself with before the training program starts.

How to Use This Plan

This document is your complete boarding roadmap. Here is how to work through it:

Step 1	Complete Part 1 (Pre-Boarding Foundations) BEFORE your first day. Estimated time: 20–30 hours over 2 weeks. You must be able to show and push work to your github
Step 2	On Day 1, Explain what CORS, Pydantic, and React Router do in one sentence each.
Step 3	Read through Part 2 (AI Concepts Brush-Up — 8 topics) before joining. These are concepts only — no builds required. Just make sure each 'Done When' makes sense to you.
Step 4	Use the Progress Tracker at the end of each section to mark your status. Bring it to for mentor check-in on Day 1

Resource Labels: [★ Primary] = start here, required. [☆ Supplementary] = optional deeper dive. All links are free or have a free tier – if any link is not working , please use similar one searching through the internet

⚠ **Rule: The 'Done When' at the end of each task is the checkpoint. Only mark a task complete when you can genuinely satisfy that criterion — not just when you finish watching the video.**

PART 1 — PRE-BOARDING FOUNDATIONS

Complete BEFORE Day 1 · Estimated: 20–30 hours · Sent at offer acceptance

These are the technical foundations every intern must have before the AI training program starts. If you already know a topic well, do a quick skim and move on — these are flagged where optional for experienced devs.

1 Python Fundamentals *(optional for experienced devs)* — Est. 2–3h

Variables, lists, dicts, loops, functions, OOP basics, virtual environments (venv), pip. If you already write Python comfortably, do a quick skim of the OOP section and move on.

★ Primary Resources

- Programming with Mosh — Python for Beginners (6h full course)
- freeCodeCamp — Python Crash Course (4h)
- Google Crash Course on Python — Coursera (audit free)

☆ Supplementary Resources

- [Optional] Official Python Tutorial — docs.python.org
- [Optional] Corey Schafer — Python OOP Tutorials (Playlist)

Done When: You can write a Python script that reads a JSON file, filters a list of dicts by a condition, and prints the result. You can create and activate a virtual environment.

2 Git & GitHub Basics — Est. 2h

git init, clone, add, commit, push, pull, branching, merging, pull requests. Create a GitHub repo and push code. We use Git for everything at Tachyon.

★ Primary Resources

- freeCodeCamp — Git and GitHub Crash Course (1h)
- Traversy Media — Git & GitHub Crash Course for Beginners

☆ Supplementary Resources

- [Optional] GitHub Docs — Getting Started
- [Optional] Interactive Git Tutorial — learngitbranching.js.org

Done When: Create a GitHub repo, clone it, create a branch, add a Python file, commit with a message, push to the branch, and open a pull request.

3 HTML + CSS + JavaScript Basics — Est. 3h

DOM manipulation, event listeners, fetch API, ES6 arrow functions, promises, async/await. You need just enough JS to understand how React works under the hood.

★ Primary Resources

- freeCodeCamp — HTML & CSS Full Course (4h)
- Traversy Media — JavaScript Crash Course for Beginners (1.5h)
- freeCodeCamp — JavaScript Algorithms and Data Structures

☆ Supplementary Resources

- [Optional] MDN Web Docs — Learn Web Development
- [Optional] JavaScript.info — Modern JavaScript Tutorial

Done When: Write a plain HTML page with a button that calls fetch() to a public API (e.g. JSONPlaceholder) and displays results in a div using DOM manipulation. No frameworks.

4 Docker Fundamentals — Est. 2h

What is a container vs a VM, writing a Dockerfile, docker build, docker run, docker push to Docker Hub. You need this to containerise and deploy the FastAPI backend to cloud.

★ Primary Resources

- TechWorld with Nana — Docker Crash Course for Absolute Beginners (1h)
- Docker Official — Get Started Tutorial

☆ Supplementary Resources

- [Optional] TechWorld with Nana — Docker Tutorial Full Course (3h)
- [Optional] Play with Docker — Free Browser-based Lab

Done When: Write a Dockerfile for a simple Python script, build the image, run the container, and confirm the script output appears. Push the image to Docker Hub.

5 FastAPI Introduction — Est. 2h

Install FastAPI + Uvicorn, create GET/POST/PATCH routes, define Pydantic models for request/response validation, configure CORSMiddleware, run locally, and explore the auto-generated /docs.

★ Primary Resources

- freeCodeCamp — FastAPI Course for Beginners (1h)
- Amigoscode — FastAPI Tutorial: Building RESTful APIs (1h)
- FastAPI Official Tutorial — First Steps

☆ Supplementary Resources

- [Optional] FastAPI Official Docs — Full Tutorial
- [Optional] Pydantic v2 Docs

Done When: Build a FastAPI app with at least one GET and one POST route using Pydantic models. Add CORSMiddleware allowing all origins. Open /docs in a browser and test both routes interactively.

6 ReactJS Basics — Est. 3–4h

Components, JSX, useState, useEffect, props, npm start. Focus on basic component rendering, event handling (onClick, onChange), and how to fetch data from an API and display it.

★ Primary Resources

- freeCodeCamp — React Course for Beginners (12h — watch first 3h)
- Traversy Media — React Crash Course (2h)
- React Official Docs — Learn React

☆ Supplementary Resources

- [Optional] Codevolution — React Tutorial for Beginners (Playlist)
- [Optional] Scrimba — Learn React for Free (interactive)

Done When: Build a React component that calls a local FastAPI endpoint (from item 5) on page load, stores the result in useState, and renders the data in a list.

7 React Router v7 — Est. 1–2h

BrowserRouter, Route, NavLink, Outlet, nested routes. This is how the VoltStream SPA navigates between pages (Dashboard, Analytics, Devices, Billing) without full page reloads.

★ Primary Resources

- React Router v7 — Official Getting Started
- Traversy Media — React Router v6/v7 Crash Course

☆ Supplementary Resources

- [Optional] React Router Docs — Main Concepts

Done When: Build a 3-page React app (Home, About, Contact) with a navigation sidebar using NavLink. Each page renders different content. The URL changes without a full page reload.

8 Tailwind CSS — Est. 1h

Utility-first CSS framework. Learn class-based styling: spacing, flexbox, colours, responsive breakpoints. Used in VoltStream for all UI styling instead of writing custom CSS.

★ Primary Resources

- Traversy Media — Tailwind CSS Crash Course (2h)
- Tailwind CSS Official Docs

☆ Supplementary Resources

- [Optional] Tailwind CSS Playground — live experimenter

Done When: Build something on your own that uses all above concepts and bring it for Day 1 and push it to github

Part 1 — Pre-Boarding Progress Tracker

Intern Name: _____ *Start Date:* _____ *Day 1 Target:* _____

#	Topic	Done When	Est.	Status	Date	Notes
1	**Python Fundamentals (optional for experienced)**	*Write a Python script reading JSON, filtering list of dicts*	2–3h			
2	**Git & GitHub**	*Push a branch and open a pull request on GitHub*	2h			
3	**HTML + CSS + JS Basics**	*Fetch API call in plain HTML, display results in DOM*	3h			
4	**Docker Fundamentals**	*Build + run + push a container to Docker Hub*	2h			
5	**FastAPI Introduction**	*GET + POST routes with Pydantic, CORS, live /docs*	2h			
6	**ReactJS Basics**	*React component fetching FastAPI data into useState*	3–4h			
7	**React Router v7**	*3-page SPA with NavLink sidebar, URL changes cleanly*	1–2h			
8	**Tailwind CSS**	*Styled responsive sidebar app using utility classes*	1h			

Status: Not Started | In Progress | Completed | Needs Help

PART 2 — AI CONCEPTS BRUSH-UP

Read through before joining · No builds required · Concepts only

These are the AI concepts you should be familiar with before the training program starts. You do not need to build anything here — just read, watch the linked resources, and make sure the 'Done When' makes sense to you. The actual hands-on building happens during the 6-week training after joining

This is a brush-up, not a project. If a concept is completely new to you, watch the linked video and practice wherever possible and build things and push it to github , we can talk more once you join.

1. AI Landscape

Key Concepts

- Basic AI/ML: learns patterns to predict or classify (spam filter, recommendations)
- Generative AI: creates new content (text, images, code) — powered by LLMs
- Agentic AI: GenAI + tools + goals + memory — plans and executes multi-step tasks
- Transformers: model architecture using 'attention' to weigh which words matter most
- Tokens, Embeddings, Context Window — the vocabulary every AI engineer must know

Resources

- Andrej Karpathy — Intro to Large Language Models (1h)
- 3Blue1Brown — Visual intro to Transformers (27 min)

Done When:

Explain the difference between GenAI and Agentic AI in 30 seconds without notes.

2. Daily AI Tools

Key Concepts

- Chat: Claude (claude.ai), ChatGPT, Gemini — different strengths and model families
- Coding assistants: GitHub Copilot, Cursor, Claude Code — AI-powered coding in the IDE
- Builders / Frameworks: LangChain, Vertex AI Agent Builder, AWS Bedrock Agents, MCP

Resources

- Claude — Try it free
- GitHub Copilot — Free tier for individuals

Done When:

Tried at least one tool from each category. You can describe when to use Claude vs GitHub Copilot.

3. LLM API Basics

Key Concepts

- API key: a secret token (sk-ant-...) — never commit to Git, always use .env or Secret Manager
- How it works: your code sends the key as an HTTP header + a prompt in the JSON body
- Pricing: charged per token (input + output tokens) — check before running in loops

Resources

- Anthropic API Quickstart (official docs)
- Anthropic Cookbook — example Python scripts

Done When:

Called the Anthropic API from Python and printed the model's response in your terminal.

4. Databases for AI

Key Concepts

- Postgres + pgvector: relational SQL + vector extension — one DB for both structured data and embeddings
- MongoDB: document DB — good for chat logs, JSON, flexible session data
- ChromaDB: specialised vector DB for semantic similarity search — easiest for local dev

Resources

- ChromaDB Getting Started
- What is a Vector Database? — Pinecone

Done When:

You can answer: where do I store user profiles vs chat embeddings vs search vectors?

5. RAG Pipeline

Key Concepts

- RAG = Retrieval-Augmented Generation: give the LLM your private docs before it answers
- Ingest: chunk docs → embed → store in vector DB
- Retrieve: embed question → similarity search → top-k chunks
- Generate: send chunks + question to LLM → grounded answer — only from the document

Resources

- DeepLearning.AI — Vector Databases & Embeddings (free, 1.5h)
- LangChain RAG Tutorial

Done When:

You understand the 3-step RAG flow (Ingest → Retrieve → Generate) and can explain it without notes.

6. Cloud Basics for AI

Key Concepts

- Serverless functions: Lambda (AWS) / Cloud Functions (GCP) — run code without managing servers
- Object storage: S3 (AWS) / GCS (GCP) — store PDFs, model artifacts, logs
- Deploying FastAPI: containerise with Docker → Cloud Run (GCP) or Lambda + Mangum (AWS)
- Secrets Manager: never hardcode API keys in production — use cloud secret stores

Resources

- GCP Free Tier — $300 credits to get started
- AWS Free Tier

Done When:

You know what Cloud Run and Lambda do and can name the deployment steps for a FastAPI app.

7. Agentic Frameworks + MCP

Key Concepts

- Agent = LLM + tools + planner + memory — it decides which tool to call based on the user query
- GCP track: Google ADK (Agent Development Kit) — official toolkit for ADK/Gemini agents
- AWS track: Strands Agents — lightweight Python framework backed by Bedrock
- MCP (Model Context Protocol): Anthropic's open standard for agents to call tools safely
- Webhook: a URL that receives an HTTP POST when an event occurs (e.g. Slack message sent)

Resources

- Google ADK Docs
- Strands Agents Quickstart
- MCP Official Quickstart

Done When:

You can explain what an agent is, what a tool call is, and why MCP exists — in plain English.

8. Production Readiness

Key Concepts

- Evals: test Q&A pairs that auto-score whether the agent answered correctly
- Observability: log cost (tokens × price), latency, and errors for every request
- RAG improvements: re-ranking, query rewriting, citations in every answer
- Safety: prompt injection defence, PII redaction, tool permission scoping

Resources

- DeepLearning.AI — Evaluating AI Agents (free, 2h)
- Langfuse — Open-source LLM observability (free tier)

Done When:

You understand what evals are and why observability matters before shipping an AI feature.

Part 2 — AI Concepts Brush-Up Tracker

Check off each concept once you feel comfortable explaining it. Bring this to your first day.

#	Topic	Done When	Resource	Status
1	**AI Landscape**	*Explain the difference between GenAI and Agentic AI in 30 seconds without notes.*	Andrej Karpathy	
2	**Daily AI Tools**	*Tried at least one tool from each category. You can describe when to use Claude vs GitHub Copilot.*	Claude	
3	**LLM API Basics**	*Called the Anthropic API from Python and printed the model's response in your terminal.*	Anthropic API Quickstart (official docs)	
4	**Databases for AI**	*You can answer: where do I store user profiles vs chat embeddings vs search vectors?*	ChromaDB Getting Started	
5	**RAG Pipeline**	*You understand the 3-step RAG flow (Ingest → Retrieve → Generate) and can explain it without notes.*	DeepLearning.AI	
6	**Cloud Basics for AI**	*You know what Cloud Run and Lambda do and can name the deployment steps for a FastAPI app.*	GCP Free Tier	
7	**Agentic Frameworks + MCP**	*You can explain what an agent is, what a tool call is, and why MCP exists — in plain English.*	Google ADK Docs	
8	**Production Readiness**	*You understand what evals are and why observability matters before shipping an AI feature.*	DeepLearning.AI	